

6-DAY INSTRUCTOR-LED TECHNICAL TRAINING

Core Python with AI-Integrated Development

Palani Karthikeyan — Instructor

abpalanikarthik@gmail.com

6-DAY INSTRUCTOR-LED TECHNICAL TRAINING

Core Python with AI-Integrated Development

A fast-track, hands-on bootcamp for experienced developers: master Python's object model, data structures, and standard library — while working AI coding assistants into every stage of the development lifecycle.

- Prior dev experience required
- Python 3.14 · VS Code
- AI Coding Assistant hands-on labs

```
day_01.py

developer =
Engineer(role="builder")

assistant =
configure_ai_assistant()

for day in course.days:
    day.learn()
    day.build_with(assistant)
    day.ship()
```

6-Day Agenda at a Glance

D1

Day 1

Conceptual Foundations, AI Coding Assistant Setup & Python/String Fundamentals

D2

Day 2

AI Coding Agents, Control Flow, Iterators & Core Data Structures (Lists, Tuples, Sets)

D3

Day 3

Dictionaries, Nested & Extended Data Structures, and File Handling

D4

Day 4

Modules & Packages, OOP, Comprehensions, and Exceptions/Debugging/AI-Assisted Testing

D5

Day 5

Core Libraries, Automation, and AI Tools for Developers (Copilot, Context Engineering & Adoption)

D6

Day 6

Capstone Project + Evaluation

DAY 1

Conceptual Foundations, AI Coding Assistant Setup & Python/String Fundamentals

- Conceptual Learning I: Paradigms, References & Memory Model
- Configure AI Coding Assistant
- AI-Driven Debugging, Testing, and Documentation
- Working with Strings
- Conceptual Learning II: Why Python & Industry Landscape
- Conversational Coding & Prompt Engineering for Python
- Introduction to Python

What You'll Be Able To Do

1

Explain Python's object model, name-binding semantics, and memory management well enough to reason about aliasing, copying, and garbage collection.

2

Articulate where Python fits among modern languages and why it dominates AI/ML tooling, scripting, and automation.

3

Install, configure, and productively use an AI coding assistant (Copilot or equivalent) inside an IDE for code generation, chat, and inline completion.

4

Write effective, layered prompts for Python code generation, refactoring, debugging, and enforce style/PEP 8 via system instructions.

5

Use AI assistants to generate pytest unit tests, docstrings, type hints, and documentation, while critically validating the output.

6

Read, write, and manipulate Python strings fluently — slicing, formatting, encoding, and the core string method set.

Conceptual Learning I: Paradigms, References & Memory Model

Python's runtime model: how objects, names, paradigms, and memory management actually work under the hood.

THE PROBLEM IT SOLVES

Developers coming from statically-typed, value-semantics languages (C++, Java, Go) misjudge how Python variables, copies, and memory behave.

WHY IT MATTERS

Misunderstanding references vs values causes some of the most common Python bugs (shared mutable state, unexpected aliasing).

WHERE IT'S USED

Every Python program — function arguments, list/dict mutation, caching, and multiprocessing all depend on this model.

REAL-WORLD ANALOGY

"Python variables are like sticky notes with an address on them, not boxes that hold the value itself."

Object-Based vs Object-Oriented

KEY TERMINOLOGY

- Object-based — everything is an object, but not everything supports inheritance/polymorphism (e.g. JavaScript's early model)
- Object-oriented — full support for classes, inheritance, encapsulation, polymorphism
- Python is genuinely object-oriented: even `int` and `function` are instances of classes
- `type(obj)` always returns a class; classes themselves are instances of `type`

HOW IT WORKS

- Every value in Python — `5`, `"hi"`, a function, a module — is an object with an identity, type, and value
- `isinstance(5, object)` is `True`: `object` is the root of every class hierarchy
- Custom classes get full OOP: inheritance, magic methods, operator overloading

DEVELOPER PERSPECTIVE

- Coming from Java/C++, expect **more** uniformity: no 'primitives' escape the object model
- This uniformity is why introspection (`type`, `dir`, `isinstance`) works identically everywhere

Basic Example

example.py

```
print(type(5))
print(type("hello"))
print(type(print))
print(isinstance(5, object))
print(type(int))
```

WHAT'S HAPPENING

- `type()` reveals the class behind any value, including built-ins and functions
- Functions are first-class objects — they have a type just like `int` or `str`
- `isinstance(5, object)` proves every value derives from the universal base class
- `type(int)` shows that classes themselves are instances of the metaclass `'type'`

CONSOLE OUTPUT

```
<class 'int'>
<class 'str'>
<class 'builtin_function_or_method'>
True
<class 'type'>
```

The Multiparadigm Shift

KEY TERMINOLOGY

- Procedural — sequential statements, functions, no mandatory objects
- Object-oriented — classes, inheritance, encapsulated state
- Functional — first-class functions, map/filter/lambda, avoiding mutation

HOW IT WORKS

- A single Python module can freely mix all three styles — no framework choice forces one paradigm
- `map()`, `filter()`, `functools.reduce()`, and `lambda` give functional-style pipelines
- Decorators and generators borrow ideas from functional composition

DEVELOPER PERSPECTIVE

- Unlike Java (forced OOP) or Haskell (forced functional), Python lets you pick the right tool per problem
- Idiomatic Python often favors 'flat is better than nested' procedural clarity over deep class hierarchies

Practical Example

```
practical_example.py

# same task, three paradigms
nums = [1, 2, 3, 4, 5]

# procedural
squares = []
for n in nums:
    squares.append(n * n)

# functional
squares_fn = list(map(lambda n: n * n, nums))

# object-oriented
class Squarer:
    def apply(self, values):
        return [v * v for v in values]
```

WHAT'S HAPPENING

- All three approaches produce the identical result list
- Procedural style: explicit loop and accumulator
- Functional style: map + lambda, no explicit loop variable
- OOP style: behavior wrapped in a class method for reuse/extension

CONSOLE OUTPUT

```
[1, 4, 9, 16, 25] [1, 4, 9, 16, 25] [1, 4, 9, 16, 25]
```

References: Names, Not Boxes

KEY TERMINOLOGY

- Name binding — a variable is a label pointing to an object, not a memory box holding a value
- Alias — two names bound to the same object
- `id(obj)` — returns the object's identity (CPython: its memory address)
- `is` vs `==` — `'is'` compares identity, `'=='` compares value equality

HOW IT WORKS

- Assignment (`a = b`) copies the reference, never the underlying object
- Mutating a mutable object through one name is visible through every alias
- Reassigning a name (`a = new_value`) only rebinds that name; it never affects other aliases

IMPORTANT RULES

- Function arguments are passed by object reference ('pass by assignment'), not by value or by pointer-to-pointer
- Small ints and interned strings may share identity due to caching — never rely on `'is'` for value comparison

DEVELOPER PERSPECTIVE

- This is the #1 source of 'spooky action at a distance' bugs for devs from C/C++/Java
- Think in terms of 'who else points at this object?' before mutating in place

Basic Example

```
example.py
a = [1, 2, 3]
b = a           # b is an alias, not a copy
b.append(4)
print(a)
print(a is b)
print(id(a) == id(b))

b = [9, 9]      # rebinding b does not affect a
print(a, b)
```

WHAT'S HAPPENING

- `b = a` binds `b` to the exact same list object as `a`
- Mutating through `b` (`append`) is visible via `a` — they are the same object
- `'is'` and `id()` confirm shared identity, not just equal contents
- Reassigning `b` to a new list only rebinds the name `b`; `a` is untouched

CONSOLE OUTPUT

```
[1, 2, 3, 4]
True
True
[1, 2, 3, 4] [9, 9]
```

Name Binding vs Value Semantics



Stack-Based vs Heap-Based Objects

KEY TERMINOLOGY

- Stack frame — holds a function call's local names (references), not the objects themselves
- Heap — where every Python object actually lives, regardless of type
- Frame — CPython's per-call record of local variables, tied to the call stack

HOW IT WORKS

- Unlike C/C++, Python has no stack-allocated value objects — ints, tuples, everything is heap-allocated
- Only the **reference** (a pointer) sits in the frame's local variable slot; the object itself is on the heap
- When a function returns, its frame is popped, but referenced heap objects survive if still reachable

DEVELOPER PERSPECTIVE

- There is no Python equivalent of a C 'stack-allocated struct' that dies at function exit unless heap-copied
- This is why returning a local list/dict from a function is always safe in Python

Bookkeeping: Packages & Namespaces

KEY TERMINOLOGY

- Namespace — a mapping from names to objects (e.g. a module's `__dict__`, a function's local scope)
- Module — a single .py file, itself an object with its own namespace
- Package — a directory of modules with an `__init__.py` (or namespace package), forming a namespace tree
- LEGB — Local, Enclosing, Global, Built-in: Python's name-resolution order

HOW IT WORKS

- `import` creates/binds a module object and inserts it into `sys.modules` (cached, loaded once per process)
- Each function call gets a fresh local namespace; classes and modules get persistent namespaces
- 'Namespaces are one honking great idea' — The Zen of Python

DEVELOPER PERSPECTIVE

- Avoid `from module import *` — it pollutes the global namespace and hides where names came from
- Use `import module` or `from module import name` explicitly for traceability

Shallow Copy vs Deep Copy

KEY TERMINOLOGY

- Shallow copy — new outer container, but inner objects are still shared references
- Deep copy — recursively copies the container and every nested object
- `copy.copy(obj)` — shallow copy; `copy.deepcopy(obj)` — deep copy

HOW IT WORKS

- `list(a)`, `a[:]`, and `dict(a)` all produce shallow copies of the outer container only
- Mutating a nested mutable object inside a shallow copy affects the original too
- `deepcopy()` walks the whole object graph, handling shared/cyclic references safely

IMPORTANT RULES

- Immutable elements (int, str, tuple of immutables) make shallow vs deep copy irrelevant
- Deep copy is slower and memory-heavier — use only when nested mutation independence is required

Practical Example

```
practical_example.py

import copy

original = {"user": "alice", "roles": ["admin", "editor"]}

shallow = original.copy()
shallow["roles"].append("viewer")
print("original after shallow mutation:", original["roles"])

deep = copy.deepcopy(original)
deep["roles"].append("guest")
print("original after deep mutation:", original["roles"])
print("deep copy roles:", deep["roles"])
```

WHAT'S HAPPENING

- `shallow = original.copy()` creates a new dict, but 'roles' list is still the same shared object
- Appending through the shallow copy's list mutates the original's list too
- `copy.deepcopy()` recursively clones nested containers, fully decoupling them
- Appending to the deep copy's roles list leaves the original untouched

CONSOLE OUTPUT

```
original after shallow mutation: ['admin',
'editor', 'viewer']
original after deep mutation: ['admin',
'editor', 'viewer']
deep copy roles: ['admin', 'editor',
'viewer', 'guest']
```

Deep Dive: Objects in Memory

KEY TERMINOLOGY

- `PyObject` — CPython's C-level header every object carries: type pointer + reference count
- `sys.getsizeof(obj)` — reports the shallow byte size of an object in memory
- Interning — CPython caches small ints (-5 to 256) and some string literals as singletons

HOW IT WORKS

- Every object header stores: reference count, a pointer to its type object, plus type-specific data
- `id(obj)` exposes the object's identity (its memory address in CPython specifically)
- Containers (list, dict) store pointers to their elements, not the elements inline

DEVELOPER PERSPECTIVE

- `sys.getsizeof([1,2,3])` measures only the list's own overhead, not the size of the ints it points to
- Don't assume identical values always share identity — only rely on caching for small ints/interned strings

Advanced / Developer Example

```
advanced_example.py

import sys

a = 256
b = 256
print(a is b)           # small int cache

x = 257
y = 257
print(x is y)           # not guaranteed outside the cache range

print(sys.getsizeof(42))
print(sys.getsizeof("hi"))
print(sys.getsizeof([1, 2, 3]))
```

WHAT'S HAPPENING

- Ints from -5 to 256 are pre-allocated singletons, so 'is' comparisons return True
- 257 falls outside the cached range, so identity is not guaranteed (implementation detail, don't rely on it)
- `sys.getsizeof()` reports bytes for the object's own header + data, not nested contents
- A 3-element list's reported size covers its pointer array, not the three int objects themselves

CONSOLE OUTPUT

```
True
False
28
43
88
```

The Garbage Collector

KEY TERMINOLOGY

- Reference counting — CPython's primary mechanism: each object tracks how many references point to it
- Generational GC — a supplementary collector that finds reference cycles; refcounting alone can't free
- gc module — Python's interface to inspect/tune/force the cyclic collector

HOW IT WORKS

- When an object's refcount hits zero, CPython frees it immediately (deterministic for most objects)
- Reference cycles (a -> b -> a) never hit zero refcount, so a separate generational GC scans for them
- Objects are grouped into 3 generations; young objects are collected more frequently than old ones

IMPORTANT RULES

- `gc.collect()` forces a full collection; `gc.disable()/enable()` toggle the cyclic collector
- `__del__` finalizers can complicate/delay cyclic collection — avoid relying on them for critical cleanup

DEVELOPER PERSPECTIVE

- Unlike Java's fully generational tracing GC, most Python cleanup is prompt refcounting, not a GC pause
- Use context managers ('with') for deterministic resource cleanup instead of trusting the GC's timing

Advanced / Developer Example

```
advanced_example.py

import sys, gc

a = []
print(sys.getrefcount(a))  # includes the temp arg reference

b = a
print(sys.getrefcount(a))  # one more alias -> higher count

class Node:
    def __init__(self):
        self.other = None

n1, n2 = Node(), Node()
n1.other = n2
n2.other = n1          # reference cycle
del n1, n2
```

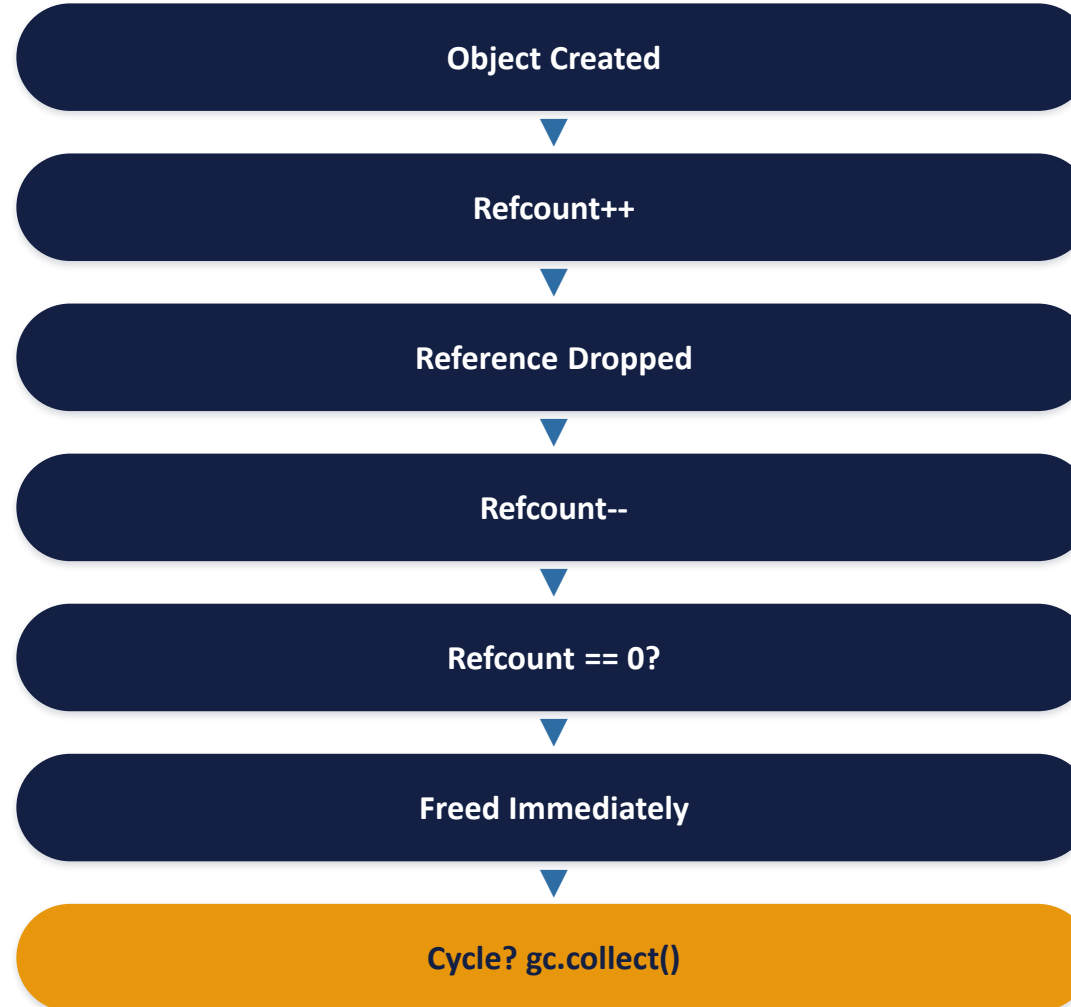
WHAT'S HAPPENING

- `sys.getrefcount()` shows the live reference count (temporarily +1 for the call argument itself)
- Adding alias `b` increases the refcount without copying the object
- `n1` and `n2` reference each other, forming a cycle that refcounting alone can never clear
- `gc.collect()` explicitly runs the cyclic collector, reclaiming the unreachable cycle

CONSOLE OUTPUT

```
2
3
objects collected: True
```

Garbage Collection Flow



Common Mistakes

✗ INCORRECT

```
def add_item(item, bucket=[]):  
    bucket.append(item)  
    return bucket
```

✓ CORRECT

```
def add_item(item, bucket=None):  
    bucket = bucket if bucket is not None else []  
    bucket.append(item)  
    return bucket
```

✗ INCORRECT

```
copy_a = original  
copy_a.append(1)
```

✓ CORRECT

```
copy_a = original.copy()  
copy_a.append(1)
```

✗ INCORRECT

```
if x is 1000:  
    ...
```

✓ CORRECT

```
if x == 1000:  
    ...
```

✗ INCORRECT

```
shallow = dict(config)  
shallow['nested']['key'] = 1
```

✓ CORRECT

```
deep = copy.deepcopy(config)  
deep['nested']['key'] = 1
```

✗ INCORRECT

```
class Cleaner:  
    def __del__(self):  
        self.conn.close()
```

✓ CORRECT

```
class Cleaner:  
    def close(self):  
        self.conn.close()  
  
# use with contextlib.closing / a context manager
```

Common Mistakes — Why It Matters

- Mistake 1: Mutable default arguments are created once and shared across every call.
- Mistake 2: Plain assignment aliases the same object instead of copying it.
- Mistake 3: 'is' checks identity; large ints outside the cache range are not guaranteed to share identity.
- Mistake 4: dict() only copies the top level; nested containers remain shared references.
- Mistake 5: `__del__` timing is not guaranteed, especially with reference cycles; use explicit cleanup instead.

Best Practices

PEP 8 & READABILITY

- Name variables for what they reference, not their storage location
- Prefer explicit `copy()`/`deepcopy()` calls over implicit aliasing when independence is required

MAINTAINABILITY

- Never use mutable objects as default function arguments
- Use `'is None'` / `'is not None'` for identity checks against singletons, `'=='` for value comparisons

PERFORMANCE

- Avoid `deepcopy()` on large object graphs in hot paths — copy only what must be independent
- Let refcounting do its job; only call `gc.collect()` when you specifically need cyclic cleanup now

Hands-on Exercise

PROBLEM STATEMENT

Write a script that demonstrates aliasing vs copying: create a list, alias it, shallow-copy it, and deep-copy a nested version, printing `id()` and contents at each step to prove which operations share memory.

INPUT

A starting list such as `data = [1, 2, [3, 4]]`

EXPECTED OUTPUT

Printed `id()` values showing alias shares id with original; shallow copy has a different outer id but shares the nested list's id; deep copy shares neither.

HINTS (no full solution here — give it a try first!)

- Use `id()` before and after each copy operation
- Mutate the nested list through each copy and observe which other variables see the change
- Use `copy.deepcopy` for the fully independent version

Conceptual Learning II: Why Python & Industry Landscape

Python's positioning as a scripting/general-purpose language — when to reach for it, when not to, and where it sits in today's AI-driven tooling landscape.

THE PROBLEM IT SOLVES

Teams pick languages by habit or hype rather than fit; experienced devs need an honest, comparative view of Python's strengths and limits.

WHY IT MATTERS

Choosing Python (or not) well up front saves rewrites and avoids fighting the language for problems it's not suited to.

WHERE IT'S USED

Startup MVPs, data/ML pipelines, automation/DevOps tooling, backend APIs, and now virtually all LLM/agent tooling.

REAL-WORLD ANALOGY

"Python is the Swiss Army knife of the language toolbox — not the fastest blade for every job, but almost always in reach and good enough."

Why Scripting? Why Python?

KEY TERMINOLOGY

- Scripting language — designed for fast iteration: no separate compile step, dynamic typing, terse syntax
- REPL — read-eval-print loop; Python's interactive shell for rapid experimentation
- 'Batteries included' — Python's philosophy of a rich standard library out of the box

HOW IT WORKS

- CPython compiles source to bytecode (.pyc) then interprets it on a virtual machine — no separate build/link step for the developer
- Dynamic typing + duck typing let you write and rerun code in seconds, ideal for glue code and prototyping
- Massive ecosystem (PyPI) means most problems already have a well-tested library

DEVELOPER PERSPECTIVE

- Scripting languages trade some raw performance and compile-time safety for developer velocity
- For a senior dev, the real win is iteration speed: test a hypothesis in the REPL before writing a single test

Basic Example

```
example.py  
  
# a 3-line script: no build step, no boilerplate main()  
import sys  
for line in sys.stdin:  
    print(line.strip().upper())
```

WHAT'S HAPPENING

- No class wrapper, no compile step — this file is directly executable as `python script.py`
- Reading from `sys.stdin` makes this usable in shell pipelines immediately
- This kind of glue/automation script is where scripting languages shine over Java/C++

CONSOLE OUTPUT

(uppercased echo of each stdin line, e.g. 'HELLO' for input 'hello')

When to Use Python (and When Not To)

WHEN TO USE PYTHON

- Data science, ML/AI pipelines, and rapid prototyping
- Automation, scripting, DevOps tooling, and glue code between systems
- Web backends (Django/Flask/FastAPI) where developer velocity outweighs raw throughput
- Any domain with strong library support (NLP, image processing, scientific computing)

WHEN NOT TO USE PYTHON

- Hard real-time systems or ultra-low-latency paths (embedded, high-frequency trading engines)
- CPU-bound, massively parallel workloads without native/C extensions (raw compute loops)
- Mobile app development or memory-constrained embedded devices
- Situations demanding compile-time type safety across a very large team without tooling discipline

DEVELOPER PERSPECTIVE

- The GIL (Global Interpreter Lock) limits true multi-threaded CPU parallelism in CPython — use multiprocessing or native extensions instead
- Python is often the 'orchestration' layer even when performance-critical inner loops are written in C/Rust (e.g. NumPy, PyTorch)

Python vs Alternatives

Language	Typing/Perf	Sweet Spot	Trade-off vs Python
Go	Static, compiled, fast	Concurrent network services, CLIs	Faster & safer concurrency, less ML/data tooling
Rust	Static, compiled, very fast	Systems, performance-critical, safety-critical	Steeper learning curve, slower iteration
Node.js (JS)	Dynamic, event-loop async	Full-stack JS, real-time web apps	Shares dynamic-typing risk, weaker data/ML ecosystem
Java	Static, JIT-compiled, verbose	Large enterprise systems, Android	More boilerplate, slower prototyping loop

Current AI Trends in Python

HOW IT WORKS

- Python is the de facto lingua franca of AI/ML — nearly every major model/training framework exposes a Python API first
- LLM tooling (OpenAI/Anthropic SDKs, LangChain, LlamaIndex) ships Python-first, often Python-only initially
- Notebook culture (Jupyter) makes Python the default language for research-to-production ML workflows

WHERE IT'S GROWING

- Agentic coding assistants, RAG pipelines, and fine-tuning workflows are almost universally scripted in Python
- MLOps tooling (experiment tracking, model serving wrappers) standardizes on Python for portability across teams

DEVELOPER PERSPECTIVE

- Even performance-critical model internals (CUDA kernels, C++ tensor ops) are wrapped in a Python-facing API
- AI coding assistants themselves are trained heavily on Python code, making Python + AI tooling a especially strong combination

Python Frameworks & SDK Landscape

WEB/API FRAMEWORKS

- Django — batteries-included, ORM + admin panel, best for full-stack monoliths
- Flask — minimal, unopinionated microframework for small services
- FastAPI — async-first, type-hint-driven, automatic OpenAPI docs — dominant for modern APIs

AI / DATA SDKS

- LangChain / LlamaIndex — orchestration frameworks for LLM apps, RAG, agents
- requests / httpx — the standard HTTP client libraries for API integration
- pandas / NumPy — the backbone of data manipulation and numerical computing

DEVELOPER PERSPECTIVE

- Framework choice today is often driven by AI/agent-tooling compatibility, not just web performance
- FastAPI's type hints make it a natural fit for AI-assisted code generation and validation

Best Practices

MAINTAINABILITY

- Choose Python when iteration speed and library ecosystem outweigh raw performance needs
- Isolate performance-critical hot paths into native extensions (C, Rust, Cython) rather than fighting Python for speed

PERFORMANCE

- Use multiprocessing or async I/O to work around the GIL for CPU-bound or I/O-bound concurrency respectively

Hands-on Exercise

PROBLEM STATEMENT

For three hypothetical projects (a real-time trading engine, an internal automation script, and an LLM-powered chatbot), decide whether Python is the right primary language and justify each decision in one sentence.

INPUT

Project descriptions: (1) sub-millisecond trading engine, (2) nightly log-cleanup automation, (3) customer-support chatbot using an LLM API.

EXPECTED OUTPUT

A short written justification per project (e.g. 'No for #1 due to GIL/latency; Yes for #2 and #3 due to iteration speed and AI SDK support').

HINTS (no full solution here — give it a try first!)

- Weigh performance needs against development speed and ecosystem support
- Consider whether the bottleneck is CPU-bound compute or I/O/orchestration
- Recall which SDKs/frameworks are Python-first

Configure AI Coding Assistant

Setting up an AI coding assistant (e.g. GitHub Copilot) inside your IDE so it can suggest, complete, and generate code as you work.

THE PROBLEM IT SOLVES

Without proper setup, developers either don't get AI suggestions at all or get poorly-scoped, low-quality ones.

WHY IT MATTERS

A correctly configured assistant becomes a genuine productivity multiplier for boilerplate, tests, and exploration.

WHERE IT'S USED

Daily coding in VS Code, PyCharm, and terminal-based agentic tools across nearly every modern engineering team.

REAL-WORLD ANALOGY

"Configuring your AI assistant is like onboarding a very fast, slightly inexperienced pair-programmer — worth the setup investment."

Anatomy of an AI Coding Assistant

KEY TERMINOLOGY

- Extension/Plugin — the IDE-side integration (e.g. GitHub Copilot extension for VS Code)
- Inline completion — ghost-text suggestions as you type, accepted with Tab
- Chat agent — a conversational panel that can read your workspace and answer/edit on request
- Context window — the code/files the assistant currently 'sees' when generating suggestions

HOW IT WORKS

- The extension sends your current file (and often open tabs/related files) as context to a hosted model
- Inline completions are generated on every keystroke pause; chat is triggered explicitly by you
- Assistants can be workspace-aware, indexing multiple files for larger refactors

DEVELOPER PERSPECTIVE

- Treat every suggestion as a draft from a junior collaborator — review before accepting, especially for security-sensitive code

Configure AI Coding Assistant

WHAT IS IT

Installing and wiring up an AI coding assistant extension so it participates in your normal edit-run-test loop inside the IDE.

WHY DEVELOPERS NEED IT

Proper setup unlocks inline completions, chat-based reasoning, and multi-file code generation without disrupting your existing workflow.



Configure AI Coding Assistant — In Practice

EXAMPLE PROMPT

In VS Code, open the Extensions panel, install 'GitHub Copilot' and 'GitHub Copilot Chat', sign in with your GitHub account, then open the Copilot Chat panel and ask: 'Scaffold a Python function that validates an email address and returns a bool.'

```
generated_by_ai.py

import re

def is_valid_email(email: str) -> bool:
    """Return True if `email` matches a basic email pattern."""
    pattern = r"^\w.+-\w+@\w-\w+\.[a-zA-Z]{2,}$"
    return re.match(pattern, email) is not None
```

Configure AI Coding Assistant — Review & Practice

BEFORE YOU TRUST THE OUTPUT — REVIEW FOR:

Correctness

Edge cases

Security

Maintainability

HANDS-ON ACTIVITY

Install the AI assistant extension in your IDE, sign in, and use the chat agent to scaffold a small validation function from a plain-English prompt.

Configuring Inline Completion & Code Generation

HOW IT WORKS

- Automatic line completion — the assistant proposes the next line(s) as ghost text while you type, generated from surrounding code context
- Inline prompts — typing a comment describing intent (e.g. '# sort dict by value descending') often triggers a matching code suggestion
- Agent mode — some assistants can plan and execute multi-step edits across files when given a higher-level task

IMPORTANT RULES

- Tab/Enter accepts a suggestion; Esc dismisses it — suggestions never apply themselves silently
- Completions are context-length limited — very large files may only get partial context

DEVELOPER PERSPECTIVE

- Write descriptive comments and clear function names first — the assistant's suggestion quality tracks your own code clarity

Basic Example

example.py

```
# Given a list of order dicts, return total revenue for completed orders  
def total_revenue(orders):  
    return sum(o["amount"] for o in orders if o["status"] == "completed")
```

WHAT'S HAPPENING

- The leading comment describing intent is exactly the kind of inline prompt that triggers strong completions
- An assistant would typically suggest this generator-expression body directly after the function signature
- Always re-read the generated condition (here: `status == 'completed'`) against your actual business rule

CONSOLE OUTPUT

(a number, e.g. 4599.50, depending on the orders list passed in)

AI Coding Assistants & IDEs Compared

Tool	Style	Strength	Consideration
GitHub Copilot (VS Code)	Inline + chat agent	Deep VS Code integration, huge adoption	Subscription required, cloud-based
Cursor.ai	AI-native editor	Multi-file agent edits built into the editor core	Separate editor to adopt, not a plugin
Windsurf	AI-native editor (Cascade agent)	Strong autonomous multi-step agent flows	Newer ecosystem, smaller plugin base
Claude Code (CLI/IDE)	Agentic terminal + IDE assistant	Deep repo-aware reasoning, strong for large refactors	More terminal-centric workflow
PyCharm + AI Assistant	IDE-native, Python-specialized	Tight integration with Python-specific refactoring tools	Heavier IDE than VS Code

Best Practices

MAINTAINABILITY

- Keep the assistant's chat scope tight (relevant files only) for more focused suggestions
- Commit before accepting large agent-driven multi-file edits so changes are easy to diff/revert

SECURITY

- Never let an assistant auto-insert credentials, tokens, or connection strings into code
- Review AI-generated dependency additions before installing — verify package legitimacy

PERFORMANCE

- Disable inline completion temporarily in extremely large files if latency becomes noticeable

Hands-on Exercise

PROBLEM STATEMENT

Install an AI coding assistant extension in your IDE (or confirm one is already configured), sign in, and use it to scaffold a Python function that deduplicates a list while preserving order — via an inline comment prompt AND via the chat agent — then compare the two results.

INPUT

A comment prompt such as '# remove duplicates from a list, preserving order' and an equivalent chat request.

EXPECTED OUTPUT

A working dedupe function (e.g. using a seen-set) generated by both interaction modes, with your notes on differences.

HINTS (no full solution here — give it a try first!)

- Try both an inline comment and a chat-panel request for the same task
- Compare readability and correctness of both generated versions
- Test with a list containing duplicates to confirm order is preserved

Conversational Coding & Prompt Engineering for Python

Structuring natural-language requests to an AI assistant so it reliably generates, refactors, or debugs correct, idiomatic Python.

THE PROBLEM IT SOLVES

Vague prompts produce generic, sometimes-wrong code; developers need a repeatable technique for getting production-quality output.

WHY IT MATTERS

Prompt quality is now a core developer skill — it directly determines how much manual rework AI-generated code needs.

WHERE IT'S USED

Daily use of chat-based assistants for scaffolding, refactoring, debugging, and multi-file changes.

REAL-WORLD ANALOGY

"Prompting an AI assistant is like writing a clear ticket for a contractor — the more specific the spec, the closer the first draft is to done."

Anatomy of an Effective Prompt

KEY TERMINOLOGY

- Role framing — telling the model to act as a specific kind of expert (e.g. 'senior Python developer')
- System instructions — persistent rules applied to every response in a session (style, constraints)
- Workspace-aware query — a prompt referencing multiple files/modules the assistant can read

HOW IT WORKS

- Specify: role, task, inputs/outputs, constraints, and desired format (function signature, tests, etc.)
- Iterative refinement — treat the first response as a draft, then follow up with corrections
- Providing existing code/context sharply improves relevance vs a cold, context-free request

DEVELOPER PERSPECTIVE

- The clearer your own mental model of the requirement, the better the prompt — prompting forces you to specify edge cases up front

Prompt Engineering: Weak -> Expert

WEAK PROMPT

Write Python code for file processing.

BETTER PROMPT

Write a Python function that reads a CSV file, validates the required columns, removes invalid rows, and returns the cleaned data as a list of dicts.

The expert prompt pins down the function signature, error handling, logging behavior, and test coverage up front, so the generated code needs far less manual correction.

Prompt Engineering: Weak -> Expert — Expert-Level Prompt

EXPERT PROMPT

Act as a senior Python developer.

Create a production-ready function `load_clean_csv(path: str, required_columns: list[str]) -> list[dict]` that:

1. Reads the CSV file at `path` using the `csv` module.
2. Raises `ValueError` if any of `required_columns` is missing from the header.
3. Skips rows with empty values in any required column, logging a warning with the row number.
4. Returns the remaining rows as a list of dicts.
5. Includes type hints, a docstring, and at least one `pytest` test covering the missing-column error case.

System Instructions for Style Enforcement

HOW IT WORKS

- System/custom instructions persist across a whole session, so PEP 8 and project conventions don't need repeating per prompt
- Typical system instruction: 'Follow PEP 8, use type hints on all functions, prefer f-strings, max line length 88'
- Project style guides (naming conventions, docstring format, import ordering) can be pasted once as a standing instruction

MULTI-FILE & WORKSPACE QUERIES

- Workspace-aware prompts can reference other files: 'Update all callers of `parse_config` in this repo to the new signature'
- Multi-file edits require explicit scope in the prompt (which files/modules are in play) to avoid unintended changes

DEVELOPER PERSPECTIVE

- Standing style instructions turn every generated snippet into something closer to your team's existing code, cutting review time

Practical Example

```
practical_example.py

# System instruction (once, at session start):
# "Follow PEP 8. Add type hints and a one-line docstring to every function."

# Prompt: "Refactor this function to be PEP 8 compliant and add type hints."

# --- before ---
def calc(a,b,op):
    if op=='add':return a+b
    elif op=='sub':return a-b

# --- after (AI output, style-enforced) ---
def calc(a: float, b: float, op: str) -> float:
    """Apply a basic arithmetic operator to two numbers."""
    if op == "add":
        return a + b
    elif op == "sub":
```

WHAT'S HAPPENING

- The standing system instruction is what added type hints and a docstring without asking again
- Spacing, quote style, and multi-line branches now match PEP 8
- The AI also added a fallback ValueError — always verify added behavior actually matches intent

CONSOLE OUTPUT

```
(no stdout — this is a refactor; behavior
verified via unit tests)
```

Prompting for Refactoring & Debugging

WHAT IS IT

Using targeted prompts to ask the assistant to refactor existing code or diagnose a bug, rather than only generate new code.

WHY DEVELOPERS NEED IT

Most real day-to-day AI usage is editing/fixing existing code, not greenfield generation — the prompting technique differs slightly.



Prompting for Refactoring & Debugging — In Practice

EXAMPLE PROMPT

Here is a function that intermittently returns stale cached data. Identify the bug, explain the root cause, and propose a fix that keeps the existing function signature.

before.py

```
_cache = {}

def get_user(user_id):
    if user_id in _cache:
        return _cache[user_id]
    user = fetch_from_db(user_id)
    _cache[user_id] = user
    return user
```

after_ai.py

```
_cache = {}
_CACHE_TTL_SECONDS = 60

def get_user(user_id):
    entry = _cache.get(user_id)
    now = time.monotonic()
    if entry and now - entry["ts"] <
_CACHE_TTL_SECONDS:
        return entry["data"]
    user = fetch_from_db(user_id)
    _cache[user_id] = {"data": user, "ts": now}
    return user
```

Prompting for Refactoring & Debugging — Review & Practice

BEFORE YOU TRUST THE OUTPUT — REVIEW FOR:

Correctness

Edge cases

Performance

Maintainability

HANDS-ON ACTIVITY

Paste a small buggy function into your AI assistant, ask it to explain the root cause before fixing it, and confirm the explanation matches your own diagnosis.

Best Practices

PEP 8 & READABILITY

- Set standing system instructions for style once per session instead of repeating them per prompt
- Ask explicitly for docstrings and type hints if your team requires them

MAINTAINABILITY

- Scope multi-file prompts explicitly — name the files/modules in play
- Always ask the assistant to explain **why**, not just paste a fix, before applying it

SECURITY

- Never paste secrets/credentials into a prompt, even to a trusted assistant

Hands-on Exercise

PROBLEM STATEMENT

Write three prompts (weak, better, expert) for the same task — generating a Python function that parses a log file and counts errors by severity — then run the expert prompt through your AI assistant and review the result.

INPUT

A task description: 'parse a log file and count entries by severity level (INFO/WARNING/ERROR) '

EXPECTED OUTPUT

Three prompt drafts of increasing specificity, plus AI-generated code from the expert prompt with your review notes on correctness and edge cases.

HINTS (no full solution here — give it a try first!)

- The expert prompt should specify function signature, input format, and return structure
- Ask the assistant to include a docstring and handle malformed lines
- Compare how much manual correction each prompt tier needed

AI-Driven Debugging, Testing, and Documentation

Using AI assistants to automatically generate unit tests, surface edge cases and vulnerabilities, and produce docstrings/type hints/READMEs.

THE PROBLEM IT SOLVES

Testing and documentation are the most frequently skipped engineering tasks under deadline pressure.

WHY IT MATTERS

AI assistants can generate a strong first draft of tests and docs in seconds, dramatically lowering the cost of doing them well.

WHERE IT'S USED

CI pipelines, pre-merge code review, onboarding documentation, and security review workflows.

REAL-WORLD ANALOGY

"An AI assistant acts like a tireless QA reviewer who reads every function and immediately asks 'what if this input is empty?'"

What AI Can (and Can't) Do Here

KEY TERMINOLOGY

- **PyTest** — Python's most widely used testing framework; AI assistants generate `test_*.py` files targeting it
- **Edge case** — an input at a boundary (empty, None, huge, malformed) likely to break naive logic
- **Race condition** — a bug from unsynchronized concurrent access to shared state

HOW IT WORKS

- Given a function, an assistant can propose pytest test cases covering typical, boundary, and error inputs
- Assistants can scan code and flag likely edge cases, injection risks, or concurrency hazards by pattern-matching against known bug classes
- Docstrings, type hints, and README sections can be generated directly from function signatures + code behavior

DEVELOPER PERSPECTIVE

- AI-suggested tests and vulnerability flags are hypotheses, not proof — they still require human judgment and often a real security review

Practical Example — Part 1

practical_example.py (continued on next slide)

```
def parse_price(raw):  
    return float(raw.replace("$", "").replace(",", ""))
```

Practical Example — Part 2

```
practical_example.py

# --- AI-assisted result: docstring + type hints + generated pytest test ---

def parse_price(raw: str) -> float:
    """Convert a formatted price string like '$1,234.50' into a float.

    Raises:
        ValueError: if `raw` cannot be converted to a float after cleanup.
    """
    cleaned = raw.replace("$", "").replace(",", "").strip()
    return float(cleaned)

# tests/test_parse_price.py
import pytest
from pricing import parse_price

def test_parse_price_basic():
    assert parse_price("$1,234.50") == 1234.50
```

WHAT'S HAPPENING

- The undocumented original had no type hints, docstring, or error contract
- AI-assisted version adds a type hint, a docstring documenting the raised exception, and an explicit `.strip()`
- Generated pytest file covers the happy path, a no-symbol input, and an invalid-input edge case
- The developer must still confirm 'free' really should raise `ValueError` per the business rule

CONSOLE OUTPUT

```
3 passed in 0.01s
```

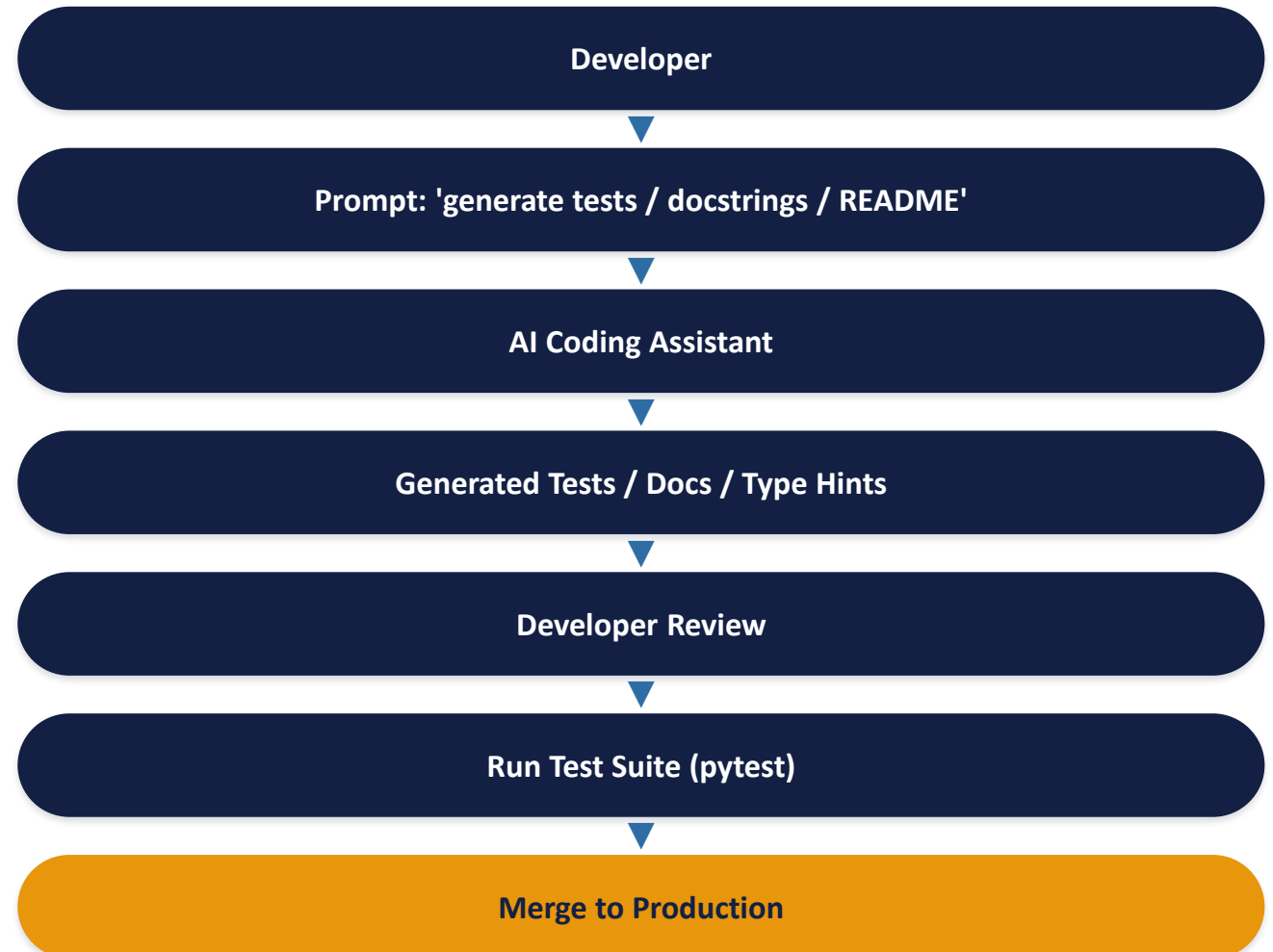
AI-Driven Debugging, Testing & Documentation

WHAT IS IT

A workflow where the AI assistant proposes tests, flags risky edge cases/vulnerabilities, and drafts documentation, while the developer reviews and integrates.

WHY DEVELOPERS NEED IT

It shifts testing/documentation from 'often skipped' to 'fast first draft plus human review', raising baseline code quality.



AI-Driven Debugging, Testing & Documentation — In Practice

EXAMPLE PROMPT

Generate pytest unit tests for this function covering normal input, empty input, None, and malformed data. Also flag any potential security issues (e.g. injection, unvalidated input) and add type hints and a docstring.

```
before.py

def run_query(user_input):
    query = "SELECT * FROM users WHERE name = '"
    + user_input + "'"
    return db.execute(query)
```

```
after_ai.py

def run_query(user_input: str):
    """Fetch users by exact name match using a
    parameterized query."""
    query = "SELECT * FROM users WHERE name =
    %s"
    return db.execute(query, (user_input,))
```

AI-Driven Debugging, Testing & Documentation — Review & Practice

BEFORE YOU TRUST THE OUTPUT — REVIEW FOR:

Security

Correctness

Edge cases

Dependencies

HANDS-ON ACTIVITY

Feed a real function from your own codebase to the assistant and ask it to generate pytest tests plus a security review; verify every flagged issue manually.

Generating Docstrings, Type Hints & READMEs

HOW IT WORKS

- Assistants infer parameter/return types from usage and code body to propose type hints even on untyped legacy code
- Docstring generation typically follows Google or NumPy style, summarizing purpose, args, returns, and raised exceptions
- README generation can summarize a module's public API by scanning function signatures and docstrings

IMPORTANT RULES

- Regenerate docstrings/type hints whenever the function signature changes — they drift silently otherwise
- Ask for exception documentation explicitly ('Raises:') since it's often omitted by default

DEVELOPER PERSPECTIVE

- Generated type hints are a strong hypothesis, not ground truth — verify against how the function is actually called elsewhere
- Treat AI-written READMEs as a first draft: confirm install/run instructions actually work before publishing

Common Mistakes

✗ INCORRECT

```
# accept AI test suite without running it
```

✓ CORRECT

```
# run `pytest -v` and confirm all AI-generated tests actually pass
```

✗ INCORRECT

```
# trust 'no vulnerabilities found' silently
```

✓ CORRECT

```
# cross-check flagged risk areas with a security linter (e.g. bandit)
```

✗ INCORRECT

```
def f(x): return x * 2 # no hint
```

✓ CORRECT

```
def f(x: int) -> int:
    """Double the given integer."""
    return x * 2
```

Common Mistakes — Why It Matters

- Mistake 1: Generated tests can contain assertions that don't match real function behavior.
- Mistake 2: AI vulnerability scanning is pattern-based, not a formal proof of safety.
- Mistake 3: Skipping AI-suggested type hints/docstrings on public functions loses discoverability for teammates.

Best Practices

MAINTAINABILITY

- Always run AI-generated tests locally before committing them
- Keep docstrings and type hints in sync when the function signature changes later

SECURITY

- Treat AI-flagged vulnerabilities as leads for investigation, not confirmed findings
- Never accept AI-suggested query/string construction that concatenates untrusted input into SQL/shell commands

PERFORMANCE

- Ask the assistant to include at least one performance/edge-case test for functions on hot paths

Hands-on Exercise

PROBLEM STATEMENT

Take an undocumented function of your own (or the `parse_price` example), ask your AI assistant to add type hints, a docstring, and generate at least 3 pytest tests including one edge case, then run the tests.

INPUT

Any small, currently undocumented Python function with at least one edge case worth testing.

EXPECTED OUTPUT

A documented, type-hinted function plus a passing pytest test file with 3+ test cases.

HINTS (no full solution here — give it a try first!)

- Explicitly ask for an edge case and an error case, not just the happy path
- Run ``pytest -v`` to confirm the generated tests actually pass
- Manually verify the docstring's claimed behavior matches the real code

Introduction to Python

Python is a high-level, dynamically-typed, interpreted language emphasizing readability and rapid development.

THE PROBLEM IT SOLVES

Experienced devs need the Python-specific syntax, tooling, and conventions layer — not general programming concepts they already know.

WHY IT MATTERS

Getting comfortable with Python's core syntax and tooling is the prerequisite for everything else in this course.

WHERE IT'S USED

Scripting, web backends, data/ML, automation — effectively every domain of modern software.

REAL-WORLD ANALOGY

"Learning Python's surface syntax is like learning the local dialect of a language family you already speak fluently."

Python Implementations

KEY TERMINOLOGY

- CPython — the reference implementation, written in C; what 'Python' means by default
- PyPy — a JIT-compiled implementation, often much faster for long-running pure-Python code
- Jython / IronPython — Python running on the JVM / .NET CLR respectively, for interop with those platforms
- Stackless Python — a CPython variant removing the C call stack, enabling lightweight microthreads

DEVELOPER PERSPECTIVE

- This course and virtually all production Python work targets CPython — the version installed by default from python.org

HOW IT WORKS

- All implementations aim for source-level compatibility with the Python language spec, but differ in runtime internals and library support
- Third-party C-extension packages (NumPy, etc.) may not work on non-CPython implementations

Python Implementations Compared

Implementation	Runtime	Best For
CPython	C, bytecode interpreter	Default/reference; broadest library compatibility
PyPy	JIT compiler	Long-running, CPU-bound pure-Python workloads
Jython	JVM	Interop with Java libraries/ecosystem
IronPython	.NET CLR	Interop with .NET/C# ecosystem
Stackless Python	CPython variant	Massive microthread concurrency (e.g. games)

Getting Started & Basic I/O

KEY TERMINOLOGY

- REPL — ``python`` or ``python3`` launches the interactive interpreter
- `input()` — reads a line from stdin as a string
- `print()` — writes values to stdout (or another stream) with configurable separators/terminators

HOW IT WORKS

- `input()` blocks until Enter is pressed, then returns everything typed (without the trailing newline) as a str
- `print()` accepts any number of positional arguments and joins them with `sep` before writing end

IMPORTANT RULES

- `input()` always returns a str — you must explicitly convert (`int()`, `float()`) for numeric input
- `print()`'s keyword arguments: `sep` (between values, default `' '`), `end` (line terminator, default `'\n'`), `file` (output stream, default `sys.stdout`)

Syntax

```
syntax.py

name = input("Enter your name: ")
print("Hello", name, sep=", ", end="!\n")

import sys
print("logged", file=sys.stderr)
```

HOW IT READS

- `input(prompt)` displays the prompt and returns the typed line as a `str`
- `sep` changes the default space between `print`'s positional arguments
- `end` replaces the default trailing newline — here producing 'Hello, <name>!'
- `file` redirects output to any writable stream, e.g. `sys.stderr` for logging-style output

Basic Example

example.py

```
print("a", "b", "c")
print("a", "b", "c", sep="-")
print("no newline", end="")
print(" continues here")
```

WHAT'S HAPPENING

- Default sep is a single space, default end is a newline
- Overriding sep changes only the join character between arguments
- Overriding end to empty string suppresses the newline, letting the next print continue the same line

CONSOLE OUTPUT

```
a b c
a-b-c
no newline continues here
```

Variables, Statements & Mutability Overview

KEY TERMINOLOGY

- Statement — a complete instruction (assignment, import, if, etc.); no semicolons required
- Immutable types — int, float, str, tuple, frozenset, bool — cannot be changed in place
- Mutable types — list, dict, set, and custom objects by default — can be changed in place

HOW IT WORKS

- No variable declarations or type annotations required — a name is bound on first assignment
- Indentation (not braces) defines block structure; consistent spacing is syntactically required
- Multiple assignment and tuple unpacking are idiomatic: `a, b = 1, 2`

DEVELOPER PERSPECTIVE

- Mutable vs immutable is covered in full depth on Day 2+ (lists/dicts) — for now, just know int/str/tuple never change in place

Basic Example

```
example.py

a, b = 1, 2
a, b = b, a          # idiomatic swap, no temp variable
print(a, b)

s = "hello"
# s[0] = "H"          # would raise TypeError: str is immutable
lst = [1, 2, 3]
lst[0] = 99           # OK: list is mutable
print(s, lst)
```

WHAT'S HAPPENING

- Tuple unpacking swaps `a` and `b` in one statement without a temporary variable
- Strings are immutable — indexed assignment would raise `TypeError` (shown commented out)
- Lists are mutable — in-place element assignment is allowed

CONSOLE OUTPUT

```
2 1
hello [99, 2, 3]
```

Built-ins, Style & Introspection

BUILT-IN FUNCTIONS & MODULES

- Always-available built-ins: `len()`, `range()`, `print()`, `input()`, `sorted()`, `sum()`, `zip()`, `enumerate()`
- Standard library modules (import required): `os`, `sys`, `math`, `json`, `re`, `datetime`, `collections`

STYLE & CONVENTIONS

- PEP 8 — the official style guide: `snake_case` for functions/variables, `PascalCase` for classes, 4-space indents
- Naming: modules/functions `lowercase_with_underscores`, constants `UPPER_CASE`

INTROSPECTION

- `type(obj)` — returns an object's class
- `dir(obj)` — lists all attributes/methods available on an object

Practical Example

```
practical_example.py

config = {"host": "localhost", "port": 8080}

print(type(config))
print("keys" in dir(config))
print([attr for attr in dir(config) if not attr.startswith("_")][:5])
```

WHAT'S HAPPENING

- `type()` confirms `config` is a dict instance
- `dir(config)` lists every attribute/method dict supports, including 'keys'
- Filtering out dunder methods gives a readable list of the public API to explore interactively

CONSOLE OUTPUT

```
<class 'dict'>
True
['clear', 'copy', 'fromkeys', 'get',
'items']
```

Common Mistakes

✗ INCORRECT

```
age = input("Age: ")
if age > 18:
    ...
```

✓ CORRECT

```
age = int(input("Age: "))
if age > 18:
    ...
```

✗ INCORRECT

```
if x == 5:
    print('five')
```

✓ CORRECT

```
if x == 5:
    print('five')
```

✗ INCORRECT

```
def calcTotal(Items):
```

✓ CORRECT

```
def calc_total(items):
```

Common Mistakes — Why It Matters

- Mistake 1: `input()` always returns a `str`; comparing `str` to `int` raises `TypeError`.
- Mistake 2: Inconsistent/insufficient indentation breaks Python's block structure (use 4 spaces consistently).
- Mistake 3: PEP 8 expects `snake_case` for functions/variables, not `camelCase`.

Best Practices

PEP 8 & READABILITY

- Use snake_case for variables/functions, PascalCase for classes, UPPER_CASE for constants
- Use 4 spaces per indentation level, never tabs, never mix the two

MAINTAINABILITY

- Convert input() results to the correct type immediately, don't defer the conversion

Hands-on Exercise

PROBLEM STATEMENT

Write a script that prompts the user for their name and age, converts age to int, and prints a formatted greeting on one line using print()'s sep/end parameters. Then use type() and dir() to explore the type of the age variable.

INPUT

Interactive input, e.g. name='Sam', age='30'

EXPECTED OUTPUT

```
Hello, Sam - you are 30 years old!  
<class 'int'>
```

HINTS (no full solution here — give it a try first!)

- Remember input() returns a str — convert age explicitly
- Use sep and end to control formatting without string concatenation
- Call dir(age) and filter out dunder attributes for a cleaner list

Working with Strings

Python's str type is an immutable sequence of Unicode code points, with a rich built-in method set and multiple literal forms.

THE PROBLEM IT SOLVES

Devs from languages with mutable char arrays or byte-oriented strings need to relearn immutability and Unicode-by-default semantics.

WHY IT MATTERS

Strings are involved in almost every program — parsing, formatting, logging, I/O — mastering them pays off immediately.

WHERE IT'S USED

Log parsing, API payloads, file I/O, templating, user-facing text everywhere.

REAL-WORLD ANALOGY

"A Python string is like a sealed, read-only sequence of characters — any 'edit' actually produces a brand-new sequence."

String Overview & Literal Forms

KEY TERMINOLOGY

- `str` — immutable sequence of Unicode code points
- Single/double quotes — interchangeable: `'text' == "text"`
- Triple-quoted strings — `"""..."""` or `'''...'''`, span multiple lines, used for docstrings
- Raw strings — `r"..."` disables backslash-escape processing (useful for regex, Windows paths)

HOW IT WORKS

- Choose single vs double quotes based on which quote character appears inside the string
- f-strings (`f"..."`) embed expressions directly: `f"{name} is {age}"`
- Strings are immutable — every 'modifying' method returns a new `str` object

DEVELOPER PERSPECTIVE

- Prefer f-strings over %-formatting or `.format()` for new code — clearer and faster

Syntax

 syntax.py

```
single = 'hello'
double = "world"
multi = """line one
line two"""
raw = r"C:\new_folder\test"
name, age = "Ada", 36
greeting = f"{name} is {age} years old"
```

HOW IT READS

- Single and double quotes are fully interchangeable for simple strings
- Triple-quoted strings preserve embedded newlines literally
- Raw strings (r"(...)") treat backslashes literally — no escape sequences are processed
- f-strings evaluate {expr} inline at runtime, formatting the result into the string

Indexing & Slicing

KEY TERMINOLOGY

- Index — 0-based position; negative indices count from the end (-1 is the last character)
- Slice — `s[start:stop:step]` extracts a substring; stop is exclusive

HOW IT WORKS

- `s[::-1]` reverses a string using a step of -1
- Omitting start/stop defaults to the beginning/end of the string respectively
- Slicing never raises `IndexError` even out-of-range — it just clamps to valid bounds

DEVELOPER PERSPECTIVE

- Slicing always returns a new str — even `s[:]` copies rather than aliasing (unlike list slicing's shallow-copy caveats)
- Reach for slicing over manual loops/indices whenever extracting a substring — it's both idiomatic and fast

Basic Example

```
example.py  
  
s = "Hello, Python!"  
print(s[0])  
print(s[-1])  
print(s[7:13])  
print(s[::-1])  
print(s[::2])
```

WHAT'S HAPPENING

- `s[0]` gets the first character; `s[-1]` gets the last via negative indexing
- `s[7:13]` slices from index 7 up to (not including) index 13
- `s[::-1]` reverses the whole string with a negative step
- `s[::2]` takes every second character starting from the beginning

CONSOLE OUTPUT

```
H  
!  
Python  
!nohtyP ,olleH  
Hlo yhn
```

The 'in' Operator, len(), and Core Methods

KEY TERMINOLOGY

- `in` — membership test: `"py" in "python" -> True`
- `len(s)` — number of characters (code points) in the string
- Core methods — `.upper()`, `.lower()`, `.replace()`, `.index()`, `.find()`, `.strip()`, `.split()`, `.join()`

HOW IT WORKS

- All string methods return new strings/lists — none mutate the original
- `.index()` raises `ValueError` if not found; `.find()` returns `-1` instead
- `.split(sep)` breaks a string into a list; `"sep".join(list)` does the reverse

DEVELOPER PERSPECTIVE

- Prefer `.find()` when a missing substring is a normal case; `.index()` when it should be an error

Practical Example

```
practical_example.py

log_line = "  2024-05-01 ERROR  connection refused  "

cleaned = log_line.strip()
print("error" in cleaned.lower())

parts = cleaned.split()
print(parts)

rejoined = "|".join(parts)
print(rejoined)

print(cleaned.replace("ERROR", "WARNING"))
print(cleaned.index("ERROR"))
```

WHAT'S HAPPENING

- `.strip()` removes leading/trailing whitespace without touching internal spacing
- `'in'` with `.lower()` gives a case-insensitive substring check
- `.split()` with no args splits on any run of whitespace into a list of tokens
- `'|'.join(parts)` reconstructs a delimited string from that list
- `.replace()` and `.index()` demonstrate non-mutating transformation and substring search

CONSOLE OUTPUT

```
True
['2024-05-01', 'ERROR', 'connection',
'refused']
2024-05-01|ERROR|connection|refused
2024-05-01 WARNING  connection refused
11
```

Special String Features & Built-in Modules

SPECIAL FEATURES

- String multiplication: `"ab" * 3` -> `"ababab"`
- String concatenation via `'+'` or implicit adjacent-literal concatenation: `"a"`
`"b"` -> `"ab"`
- Chained method calls: `s.strip().lower().replace(" ", "_")`

BUILT-IN MODULES FOR STRING HANDLING

- `string` — constants like `string.ascii_letters`, `string.digits`, `string.punctuation`
- `textwrap` — wrapping/indenting/dedenting long text blocks for display
- `re` — regular expressions for pattern matching, far beyond basic str methods

DEVELOPER PERSPECTIVE

- Reach for str methods first; only bring in `re` when you truly need pattern matching, not fixed substrings

Advanced / Developer Example

```
advanced_example.py

import string, textwrap

print(string.ascii_lowercase)
print(string.digits)

paragraph = "Python is a widely used general purpose programming language."
wrapped = textwrap.fill(paragraph, width=20)
print(wrapped)
```

WHAT'S HAPPENING

- `string.ascii_lowercase` and `string.digits` are ready-made character-set constants
- `textwrap.fill()` reflows a long string into lines no wider than the given width
- Useful for CLI output formatting or generating readable reports without manual line-breaking

CONSOLE OUTPUT

```
abcdefghijklmnopqrstuvwxyz
0123456789
Python is a widely
used general purpose
programming language.
```

Unicode Strings vs Bytes

KEY TERMINOLOGY

- `str` — text: a sequence of Unicode code points (Python 3's default string type)
- `bytes` — a sequence of raw 8-bit values, used for binary data and network/file I/O
- Encoding — converting `str` -> `bytes` using a codec (e.g. UTF-8)
- Decoding — converting `bytes` -> `str` using the same codec

HOW IT WORKS

- `s.encode("utf-8")` turns a `str` into `bytes`; `b.decode("utf-8")` turns `bytes` back into a `str`
- A `bytes` literal is written with a `b`-prefix: `b"hello"`
- Mixing `str` and `bytes` in the same operation (e.g. concatenation) raises `TypeError`

DEVELOPER PERSPECTIVE

- All network sockets and many file modes deal in `bytes` — decode explicitly at the boundary, work in `str` internally

Practical Example

```
practical_example.py

text = "caf\u00e9"          # café
encoded = text.encode("utf-8")
print(encoded)
print(type(encoded))

decoded = encoded.decode("utf-8")
print(decoded, decoded == text)

# text + encoded # would raise TypeError: can't concat str to bytes
```

WHAT'S HAPPENING

- `text.encode('utf-8')` converts the Unicode str into a bytes object using UTF-8
- The printed bytes literal shows the multi-byte encoding of the accented character
- `decode('utf-8')` reverses the process, recovering an equal str
- Attempting to mix str and bytes directly raises `TypeError`, shown commented out

CONSOLE OUTPUT

```
b'caf\xc3\xa9'
<class 'bytes'>
café True
```

Common Mistakes

✗ INCORRECT

```
s = 'hello'
s[0] = 'H'
```

✓ CORRECT

```
s = 'hello'
s = 'H' + s[1:]
```

✗ INCORRECT

```
path = "C:\new\test"
```

✓ CORRECT

```
path = r"C:\new\test"
```

✗ INCORRECT

```
if s.index("x") >= 0:
    ...
```

✓ CORRECT

```
if s.find("x") >= 0:
    ...
```

✗ INCORRECT

```
data = text + encoded_bytes
```

✓ CORRECT

```
data = text.encode() + encoded_bytes
```

Common Mistakes — Why It Matters

- Mistake 1: Strings are immutable; indexed assignment raises `TypeError`.
- Mistake 2: `\n` and `\t` are interpreted as escapes unless the string is raw or the backslashes are doubled.
- Mistake 3: `.index()` raises `ValueError` when not found; `.find()` returns `-1` instead of raising.
- Mistake 4: `str` and `bytes` cannot be concatenated directly; `encode/decode` to match types first.

Best Practices

PEP 8 & READABILITY

- Prefer f-strings for interpolation over % formatting or .format()
- Be consistent with single vs double quotes across a project (PEP 8 has no strict rule, but consistency matters)

MAINTAINABILITY

- Chain string methods thoughtfully; break very long chains into named intermediate steps

PERFORMANCE

- Avoid building strings with repeated '+' in a loop; use ''.join(list_of_parts) instead

SECURITY

- Always specify an explicit encoding (e.g. 'utf-8') rather than relying on platform defaults

Hands-on Exercise

PROBLEM STATEMENT

Given a raw log line with extra whitespace and mixed case, write a script that strips it, checks for the substring 'error' case-insensitively, splits it into tokens, rejoins them with a pipe delimiter, and prints both the UTF-8 encoded bytes and the decoded string.

INPUT

```
"  2024-06-10 Error  disk quota EXCEEDED  "
```

EXPECTED OUTPUT

```
2024-06-10|Error|disk|quota|EXCEEDED  
b'2024-06-10|Error|disk|quota|EXCEEDED'  
true (matches decoded)
```

HINTS (no full solution here — give it a try first!)

- Use `.strip()`, `.lower()`, and `'in'` for the case-insensitive check
- Use `.split()` and `'|'.join()` to rebuild the tokenized string
- Use `.encode('utf-8')` and `.decode('utf-8')` to round-trip the string

Day Summary

CONCEPTS LEARNED

- Python's object model: everything is an object, references are name bindings not value boxes
- Multiparadigm programming: procedural, OOP, and functional styles coexist freely
- Shallow vs deep copy and CPython's refcounting + generational garbage collector
- Python's positioning: scripting strengths, real limits (GIL, raw compute), and its dominance in AI/ML tooling
- AI-assisted development as a full workflow: setup, prompting, debugging/testing/documentation
- Python strings as immutable Unicode sequences distinct from bytes

KEY SYNTAX

- `type()`, `isinstance()`, `id()`, `is` vs `==`
- `copy.copy()` / `copy.deepcopy()`
- `sys.getsizeof()`, `sys.getrefcount()`, `gc.collect()`
- `input()`, `print(sep=, end=, file=)`
- f-strings, raw strings (`r"..."`), triple-quoted strings
- String slicing `s[start:stop:step]`, `.strip()`/`.split()`/`.join()`/`.replace()`/`.find()`/`.index()`
- `.encode()` / `.decode()`

KEY TAKEAWAYS

- Assignment in Python binds names to objects; it never copies data implicitly
- Mutable default arguments and shallow copies are the most common aliasing bugs for newcomers
- A configured AI assistant plus well-structured prompts materially speeds up scaffolding, refactoring, testing, and docs
- AI-generated code, tests, and vulnerability flags must always be reviewed, never trusted blindly
- Python strings are immutable — every transformation produces a new string object

PRACTICAL SKILLS GAINED

- Reasoning about aliasing, copying, and memory using `id()`, `sys.getsizeof()`, and `copy`
- Installing and configuring an AI coding assistant with chat and inline completion
- Writing layered (weak/better/expert) prompts for Python code generation and refactoring
- Using AI to generate pytest tests, docstrings, and type hints, then validating the output
- Fluent string manipulation: slicing, formatting, core methods, and encoding/decoding

AI-Assisted Log Line Processor

Your team needs a small utility script that cleans and summarizes raw application log lines pulled from a legacy system, and you'll use your AI assistant to help scaffold and document it.

REQUIREMENTS

- Use your configured AI coding assistant to scaffold an initial function skeleton from a well-structured (expert-level) prompt
- Implement a function `parse_log_line(raw: str) -> dict` that strips whitespace, splits the line into date/level/message tokens, and returns them as a dict
- Demonstrate the reference/copy model: create a list of parsed dict results, make both a shallow and a deep copy of it, mutate one, and print `id()/content` evidence of which copies were affected
- Use string methods (`.strip()`, `.split()`, `.lower()`, `'in'`) to detect and count lines containing 'error' (case-insensitive)
- Add a type hint and docstring to every function, and use your AI assistant to generate at least 2 pytest tests, including one edge case (e.g. an empty or malformed line)
- Encode the final summary string to UTF-8 bytes and decode it back, printing both forms

CONSTRAINTS

- Use only the Python standard library (no third-party parsing packages)
- `parse_log_line` must not mutate its input string
- All functions must include type hints and a one-line docstring

AI-Assisted Log Line Processor — Input / Output

SAMPLE INPUT

```
raw_lines = [  
    " 2024-06-10 ERROR disk quota exceeded ",  
    "2024-06-10 INFO backup completed",  
    " 2024-06-10 error connection refused",  
]
```

EXPECTED OUTPUT

```
parsed: [{'date': '2024-06-10', 'level': 'ERROR', 'message':  
'disk quota exceeded'}, ...]  
error_count: 2  
shallow copy shares nested objects with original (proven via  
id())  
deep copy is fully independent (proven via id())  
3 passed in pytest  
b'...' (encoded bytes) and matching decoded string
```

HINTS

- Prompt your AI assistant with an expert-level prompt specifying the exact function signature and return shape first
- Use `copy.deepcopy()` only for the list of dicts you intend to mutate independently
- Use `.lower()` plus `'in'` for the case-insensitive error check, not a regex

OPTIONAL CHALLENGE

Extend the script to also flag any log line whose message contains a common security-sensitive keyword (e.g. 'password', 'token') and ask your AI assistant to suggest additional keywords to check for.